Python OOP: Four Pillars Assignment

8 programming exercises | 2 questions per pillar | Beginner to Intermediate

Assignment Instructions

Complete all eight exercises using Python. Apply appropriate classes, constructors, and descriptive method names. Demonstrate each solution with at least two objects or test cases where applicable. Submit separate Python files named Q1.py through Q8.py, along with sample console output. Use only the Python standard library.

Pillar 1: Encapsulation

Q1. Student Result Manager | Beginner

Create a Student class with a public name, protected _marks, and private __grade attributes. Initialize the student through a constructor. Implement set_marks(marks), which accepts only values from 0 to 100, along with get_marks() and get_grade(). Assign grade A for 80 to 100, B for 60 to 79, C for 40 to 59, and F below 40. Demonstrate both a valid and an invalid marks update. **Extension:** Attempt direct access to __grade and explain the result.

Q2. Secure Bank Account | Intermediate

Create a BankAccount class with a public account holder name, private __balance, and private __pin. Implement deposit(amount), withdraw(amount, pin), and get_balance(pin). Reject zero or negative deposits, withdrawals exceeding the available balance, and incorrect PINs. Demonstrate a successful deposit, a successful withdrawal, and two rejected operations. **Extension:** Explain why double underscores do not provide true security.

Pillar 2: Inheritance

Q3. Employee Roles | Beginner

Create a parent class Employee that stores a name and employee ID and provides display_details(). Derive Developer (programming language) and Manager (team size). Use super() to initialize inherited attributes. Override display_details() in each child class to include both shared and role-specific information. Create and test one object of each type.

Q4. Vehicle Rental System | Intermediate

Create a base class Vehicle with brand, model, and daily rental rate. Implement calculate_rent(days). Derive Car, which adds a fixed insurance fee of Rs. 1,000 per rental, and Bike, which applies a 10% discount to the base rental amount. Override calculate_rent() and reuse the parent implementation with super(). Reject rental durations of zero or fewer days. Test both vehicle types for a three-day rental.

Pillar 3: Polymorphism

Q5. Notification Dispatcher | Beginner

Create EmailNotification, SMSNotification, and PushNotification classes. Each must implement send(message) with output specific to its channel. Write a single function notify_all(notifications, message) that iterates through different notification objects and invokes send(). Add a WhatsAppNotification class without changing the dispatcher. Explain where polymorphism occurs.

Q6. Checkout Discount Calculator | Intermediate

Create RegularCustomer, PremiumCustomer, and VIPCustomer classes, each implementing final_price(amount). Apply discounts of 5%, 10%, and 20%, respectively. Write print_receipt(customer, amount) so that it works with any customer type. For a bill of Rs. 10,000, show the final amount for each type. Reject negative bill amounts. **Extension:** Add GoldCustomer with a 15% discount without modifying print_receipt().

Pillar 4: Abstraction

Q7. Shape Calculator | Beginner

Use ABC and @abstractmethod to define an abstract Shape class with area() and perimeter(). Implement both methods in Circle and Rectangle. Accept dimensions through constructors and use pi = 3.14159. Test a circle with radius 5 and a rectangle with width 4 and height 6. Attempt to instantiate Shape directly and explain the outcome.

Q8. Payment Gateway | Intermediate

Define an abstract PaymentMethod class with abstract methods pay(amount) and refund(amount). Implement CardPayment, WalletPayment, and BankTransferPayment. Each method must display the payment method, transaction type, and amount. Validate that transaction amounts are positive. Write process_transaction(method, amount) to call pay() for any supported payment method. **Extension:** Add a new payment method without changing process_transaction().

Submission Checklist

Include all eight Python files and sample outputs. Ensure input validation is handled where required and each OOP principle is clearly demonstrated. Use meaningful class and method names and keep code readable.